



1. Problem Title & Link

Title: LeetCode 567 : Permutation in String

Link: [LeetCode 567 - Permutation in String](#)

2. Problem Statement (Short Summary)

You are given two strings:

s1

s2

Return true if s2 contains a permutation of s1.

A permutation means:

- Same characters
- Same frequencies
- Different order allowed

In simple terms:

Check whether any substring of s2 has exactly the same characters as s1.

3. Examples (Input → Output)

Example 1

Input:

s1="ab"

s2="eidbaooo"

Output:

true

Explanation:

Substring:

"ba"

is a permutation of:

"ab"

Example 2

Input:

s1="ab"

s2="eidboaoo"

Output:

false



Explanation:

No substring has the same frequency as ab.

Example 3

Input:

s1="adc"

s2="dcda"

Output:

true

Explanation:

"dca"

is a permutation of:

"adc"

4. Constraints

$1 \leq s1.length, s2.length \leq 10^4$

s1 and s2 contain lowercase English letters

Special restrictions:

- Must avoid generating all permutations
- Target:

$O(n)$

solution

5. Core Concept (Pattern / Topic)

Sliding Window + Frequency Counting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate all permutations of s1.

For each permutation:

- Search inside s2

Time:

$O(n!)$

Extremely slow.

Better Approach



Observation:

If two strings are permutations:

Character frequencies must match

Instead of generating permutations:

1. Store frequency of s1
2. Create a window of size len(s1) in s2
3. Compare frequencies
4. Slide window

Optimized Thinking

Maintain:

s1Count[26]

windowCount[26]

As the window moves:

- Add new character
- Remove old character
- Compare frequency arrays

If equal:

return true

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

s1="ab"

s2="eidbaooo"

Window size:

2

Traversal:

ei

id

db

ba

ao

oo

oo

Frequency match:

ba



a→1

b→1

Same as:

ab

Return:

true

8. Pseudocode (Language Independent)

create s1Count[26]

create windowCount[26]

store frequencies of s1

store first window frequency

if arrays equal:

 return true

for i from length(s1) to length(s2)-1:

 add current character

 remove old character

 if arrays equal:

 return true

return false

9. Code Implementation

Python

class Solution:

 def checkInclusion(self, s1, s2):



```
if len(s1) > len(s2):
    return False

s1Count = [0] * 26
windowCount = [0] * 26

for c in s1:
    s1Count[ord(c)-ord('a')] += 1

for c in s2[:len(s1)]:
    windowCount[ord(c)-ord('a')] += 1

if s1Count == windowCount:
    return True

left = 0

for right in range(len(s1), len(s2)):

    windowCount[
        ord(s2[right])-ord('a')
    ] += 1

    windowCount[
        ord(s2[left])-ord('a')
    ] -= 1

    left += 1

if s1Count == windowCount:
    return True

return False
```

Java

```
class Solution {
```



```
public boolean checkInclusion(
    String s1,
    String s2) {

    if(s1.length()>s2.length())
        return false;

    int[] s1Count=new int[26];
    int[] windowCount=new int[26];

    for(char c:s1.toCharArray())
        s1Count[c-'a']++;

    for(int i=0;i<s1.length();i++)
        windowCount[
            s2.charAt(i)-'a'
        ]++;

    if(Arrays.equals(
        s1Count,
        windowCount))
        return true;

    int left=0;

    for(int right=s1.length();
        right<s2.length();
        right++){

        windowCount[
            s2.charAt(right)-'a'
        ]++;

        windowCount[
            s2.charAt(left)-'a'
        ]--;
```



```
        left++;

        if(Arrays.equals(
            s1Count,
            windowCount))
            return true;
    }

    return false;
}
```

10. Time & Space Complexity

Time Complexity:

$O(n)$

Reason:

- Sliding window traverses string once
- Comparing fixed-size arrays:

26 elements

takes constant time

Space Complexity:

$O(1)$

Reason:

Frequency arrays always contain 26 elements.

11. Common Mistakes / Edge Cases

1. Forgetting:

$\text{len}(s1) > \text{len}(s2)$

Case

2. Not removing left character while sliding

Wrong:

Window keeps growing

3. Generating permutations

Very expensive:

$O(n!)$



4. Using HashMap unnecessarily for lowercase letters
Array is faster.

12. Detailed Dry Run (Step-by-Step Table)

Input:

s1="ab"

s2="eidbaooo"

Window	Frequency Match
ei	No
id	No
db	No
ba	Yes

Output:

true

13. Common Use Cases (Real-Life / Interview)

Anagram detection

Pattern matching systems

Search optimization

Fraud detection through pattern recognition

14. Common Traps (Important!)

1. Window size changing incorrectly
2. Forgetting to decrement old character frequency
3. Using sorting repeatedly

Sorting every window:

$O(n \cdot k \log k)$

is expensive

15. Builds To (Related LeetCode Problems)

Progression:

- LC 567 → Permutation in String
- LC 438 → Find All Anagrams in a String
- LC 3 → Longest Substring Without Repeating Characters



- LC 76 → Minimum Window Substring
- LC 424 → Longest Repeating Character Replacement

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Generate permutations	$O(n!)$	$O(n)$	Impossible for large input
Sort every window	$O(n * k \log k)$	$O(k)$	Repeated sorting
Sliding Window + Frequency Array	$O(n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

A permutation contains identical character frequencies. The sliding window always maintains a substring of size $\text{len}(s1)$, and frequency arrays efficiently verify whether the current window matches $s1$.

18. Variations / Follow-Up Questions

1. Return all matching indices
2. Support uppercase letters
3. Support Unicode characters
4. Find longest permutation match
5. Solve using HashMap instead of arrays
- 6.